NEW Et si on parlait pSEO ?

laConsole

Cheatsheet : JavaScript

CheatSheet JavaScript : Guide complet, résumé, aide-mémoire, tutoriel et conseils clés pour apprendre et maîtriser JavaScript efficacement et rapidement.

MAJ en nov. 2024 Intermédiaire 9 chapitres

Cours Cheatsheet

Commencer

CHEATSHEETS / JAVASCRIPT

Recherche...

Lier du JS

```
<script src="path/to/app.js"></script>
```

- Placer dans le `<head>` ou avant la fin du `<body>`
- Si JS manipule le DOM : linker avant la fin du `<body>`

Commentaire

```
// Je suis un commentaire

/* Je suis un commentaire
sur plusieurs lignes */
```

Debug

```
// Apparaît en blanc
console.log("Message");
// Apparaît en rouge avec une icône "danger"
console.error("Message d'erreur");
// Apparaît en orange avec une icône "warning"
console.warn("Message d'avertissement");
// Apparaît en blanc avec une icône "info"
console.info("Message d'information");
```

`console.clear()` : efface les messages de la console

Variable

Case mémoire stockant une donnée.

```
let firstname = "Toto";
firstname = "Tutu"; // Ecrase la valeur précédente

// Non modifiables
const age = 32;
const premium = false;
```

- Types principaux : texte, nombre et booléen
- Optimisation : utiliser le moins possible `var`
- Privilégier `const` sur `let` autant que possible

Objet

Structure de données organisée et encapsulée.

```
// Déclaration
const user = {
    firstname: 'John',
    lastname: 'Doe',
    age: 30,
    premium: false,
    address: {
        street: '123 rue des Pyrénées',
        city: 'Bordeaux',
        country: 'France'
    },
    hobbies: ['lecture', 'voyage', 'musique'],
    happyBirthday: function() {
        this.age += 1;
    }
};

// Utilisation
user.happyBirthday();
console.log(user.age); // 31
```

- Un objet peut posséder des données et/ou des fonctions
- `this` fait référence à l'objet en question

Tableau

Structure de données regroupant un ensemble de valeurs.

```
// Tableau unidimensionnel
const fruits = ["Salade", "Tomate", "Oignon"];
console.log(fruits[0]); // "Salade"

// Tableau multidimensionnel
const fruits = [
    ["Salade", 1.90],
    ["Tomate", 2.75],
    ["Oignon", 1]
];
console.log([1][1]);  // 2.75
```

Premier élément d'un tableau à l'indice `0`

Opération

```
let a = 4;
let b = 7;

let addition = a + b; // Addition (11)
let soustraction = b - a; // Soustraction (3)
let multiplication = a * b; // Multiplication (28)
let division = b / a; // Division (1.75)
let modulo = b % a; // Reste de la division (3)
```

- Si `nombre % 2 = 0` => pair et si `nombre % 2 = 1` => impair
- Opérations complexes => fonctions `sqrt()`, `exp()` ...

Concaténation

Joindre plusieurs chaînes de caractères et/ou variables.

```
let firstname = "Toto";
console.log("Hello " + firstname);
console.log(`Hello ${firstname}`); // Syntaxe alternative
```

Multiples concaténations : privilégier les backticks ` `` ` avec `${}`

If, Else

Conditionne l'exécution d'un script.

```
let dice = 5;
if (dice == 1) {
    // Il vaut mieux relancer le dé...
} else if (dice > 1 && dice <= 3) {
    // C'est mieux ...
} else if (dice > 3 && dice <= 5) {
    // Pas mal !
} else {
    // Super !
}
```

- Opérateurs de comparaison : `==`, `===`, `!=`, `!==`, `>`, `>=`, `<` et `<=`
- Opérateurs logiques : `!`, `&&` et `||`
- Il est possible d'imbriquer des conditions entre elles

Switch

Conditionne l'exécution d'un script.

```
let fruit = "Banane";
switch (fruit) {
    case 'Banane':
        console.log(`${fruit} (1.93 € / kg)`);
        break;
    case 'Pomme':
        console.log(`${fruit} (2.62 € / kg)`);
        break;
    case 'Mangue':
    case 'Papaye':
        console.log(`${fruit} (6.17 € / kg)`);
        break;
    default:
        console.log(`Désolé, nous ne vendons pas de ${fruit}`);
}
```

- `case` déclenché si valeur strictement égale (`===`) à l'expression du `switch`
- Plusieurs `case` à la suite signifient « ou »
- `break` termine l'instruction
- `default` est exécuté si aucune correspondance avec les clauses `case`

For

Répète une instruction plusieurs fois.

```
for (let i=0; i<=10; i++) {
    // 11 tours de boucle...
}
```

- `let i=0` : point de départ
- `i<=10` : point d'arrivée
- `i++` : pas (ou « step ») - raccourci pour `i=i+1`
- Alternative `forEach()` utile pour boucler sur des tableaux
- Il est possible d'imbriquer des boucles entre elles

While

Répète une instruction tant qu'une condition est vraie.

```
let n = 0;

while (n < 10) {
  n++;
}

console.log(n); // 10
```

Il est possible d'imbriquer des boucles entre elles

Fonction

Une portion de code isolée exécutable sur demande.

```
// Déclaration
function sayHello(firstname) {
    return "Hello " + firstname + " !";
}

// Appel
sayHello("Toto"); // "Hello Toto !"
```

- `firstname` : paramètre de la fonction
- `return ...` : valeur retournée par la fonction (facultatif)
- `"Toto"` : argument (valeur transmise à un paramètre)

Sélecteurs

Ciblage d'éléments HTML.

```
const element = document.querySelector("#popup"); // Sélection unique
const elements = document.querySelectorAll("a"); // Sélection multiple
```

- `querySelectorAll()` et `querySelector()` prennent en argument un sélecteur CSS
- Alternatives : `getElementsByTagName()`, `getElementsByClassName()` et `getElementById()`
- Sauvegarder la sélection dans une constante

Manipuler la structure HTML

```
// Récupération d'éléments du DOM
console.log(element.textContent); // Textuel
console.log(element.innerHTML); // HTML

// Modification du DOM
element.textContent = "Nouveau texte"; // Textuel
element.innerHTML = "<h1>Nouveau titre</h1>"; // HTML

// Création d'éléments HTML / Noeuds textuels
const p = document.createElement("p");
const text = document.createTextNode("Hello !");

// Insertion d'éléments dans le DOM
element.prepend(elementToInsert); // Premier enfant de "element"
element.append(elementToInsert); // Dernier enfant de "element"
element.before(elementToInsert); // Avant "element"
element.after(elementToInsert); // Après "element"

// Suppression d'éléments du DOM
element.removeChild(elementToRemove);
```

- `querySelectorAll()` et `querySelector()` prennent en argument un sélecteur CSS
- Alternatives : `getElementsByTagName()`, `getElementsByClassName()` et `getElementById()`

Manipuler les attributs HTML

```
// Retourne la source de l'élément
element.getAttribute("src");
// Modifie la source de l'élément
element.setAttribute("src", "https://laconsole.dev");
```

Manipuler les classes HTML

```
// Retourne la liste des classes
element.classList;
// Ajoute les classes "bg-dark" et "text-white"
element.classList.add("bg-dark", "text-white");
// Supprime la classe "active"
element.classList.remove("active");
// Ajoute la classe "active" si absente / la supprime si présente
element.classList.toggle("active");
// Vérifie si la classe "active" est présente sur l'élément
element.classList.contains("active");
// Remplace la classe "bg-dark" par "bg-light"
element.classList.replace("bg-dark", "bg-light");
```

Interagir avec les formulaires

```
// Récupère la valeur saisie dans un champ
const fieldValue = element.value;
// Redéfinit la valeur d'un champ
element.value = "Toto";
```

Manipuler le CSS

```
// Récupère la taille du texte (si définie en style inline)
const elementColor = element.style.color;
// Redéfinit la taille du texte
element.style.fontSize = "22px";
```

- Propriétés CSS doivent être écrites en `camelCase`
- Si CSS défini dans un `.css`, utiliser `getComputedStyle()` et `getPropertyValue()`

Évènements

Interagir avec la page web.

```
element.addEventListener("click", function(event) {
    // ...
    // Ici le mot-clé "this" fait référence à "element"
});
```

- Evènements courants : `click`, `key***`, `input` et `change` (voir plus)
- Evènements mobiles : `touchstart`, `touchend`, `touchcancel`, `touchleave` et `touchmove`
- `removeEventListener()` permet de supprimer un écouteur d'évènement
- L'objet `event` contient des propriétés et méthodes communes à tous les évènements

Intervalle

Exécution d'un script à intervalles de temps réguliers.

```
function hello() {
  console.log("Hello !");
}

// Affiche "Hello !" chaque seconde
const intervalID = setInterval(hello, 1000);

// clearInterval(intervalID) --> stoppe l'intervalle
```

Décompte

Exécution retardée d'un script.

```
function hello() {
  console.log("Hello !");
}

// Affiche "Hello !" au bout de 2 secondes
const timeoutID = setTimeout(hello, 2000);

// clearTimeout(timeoutID) --> arrêt anticipé
```

Cookie

Petit fichier textuel stockant dans le navigateur des données de navigation de l'utilisateur.

```
// Création
document.cookie = "nom=valeur; domain=monsite.com; path=/shop; max-age=86400";

// Modification (important de redéfinir les options à conserver)
document.cookie = 'nom=nouvelle_valeur; domain=monsite.com; max-age=86400';

// Suppression (sans préciser de valeur + max-age=0)
document.cookie = 'nom=; max-age=0';

// Lecture (tous les cookies)
console.log(document.cookie);
```

- `nom=valeur` : créé un cookie comme une paire `clé-valeur`
- `domain` : domaine dans lequel le cookie sera accessible. Si non spécifié, domaine ou sous-domaine dans lequel il a été créé uniquement. Si spécifié : sous-domaines toujours inclus
- `path` : chemin (+ chemins enfants) dans lesquels le cookie sera accessible. Si non spécifié, chemin (+ chemins enfants) dans lequel il a été créé
- `max-age` : durée de vie du cookie (en secondes)
- `expires` : date d'expiration du cookie

Callback

Une fonction passée en argument d'une autre fonction.

```
function demo(firstName, cb) {
    console.log(`Hello ${firstName}`);
    cb();
}

demo('John', () => {
    console.log('Je suis le callback');
})
```

Then, Catch, Finally

Traitement d'une fonction asynchrone retournant une Promesse.

```
fetch(`https://jsonplaceholder.typicode.com/posts`)
    .then(response => response.json())
  .then(data => console.log(data))
    .catch(error => console.error(error))
    .finally(() => console.log("Promesse traitée."))
```

- `.then()` exécuté si la `Promise` est résolue
- `.catch()` exécuté si la `Promise` est rejetée
- `.finally()` exécuté après retour de la `Promise` (résolue ou rejetée)

Async / Await

Traitement d'une fonction asynchrone retournant une Promesse.

```
async function getUser() {
    // Retourne une promesse contenant un user...
}

(async () => {
  const user = await getUser();
  console.log(user);
})();
```

- `async` permet de déclarer une fonction asynchrone
- `await` attend la résolution d'une `Promise` (utilisable uniquement dans une fonction `async`)

laConsole

laConsole, plateforme d'e-learning dédiée au développement web.

M'abonner à la newsletter

status : **200** laConsole © 2026

50